PROJECT DOCUMENTATION

Ahmad Shah

Company Management System — Frontend

A Vue 3 + Vuetify single-page admin application for managing a manufacturing/trading business: staff, customers, products, sales, finances, assets and more.

Document version 1.0

1. Overview

Ahmad Shah is an internal back-office (ERP-style) web application for running a company end-to-end. It manages people (users, staff, customers, contacts, investors), inventory and production (products, raw materials, production capacity), finances (sales, payments, expenses, bank accounts, daily transactions), company assets, and operational records (daily tasks, attendance, visa registrations, passwords).

The interface is fully **Persian/Dari and right-to-left (RTL)**. The frontend is a Single Page Application that talks to a separate **Laravel REST API** backend.

Table of contents

2. Technology Stack

Area	Technology
Framework	Vue 3 (Composition API, `<script setup>`)
UI library	Vuetify 3 (Material Design, RTL)
Build tool	Vite
Routing	vue-router with file-based routing (`vite-plugin-pages`) + layouts
State management	Pinia
HTTP client	Axios (custom instance with interceptors)
Localization	vue-i18n (Persian / English)
Notifications	vue3-toastify
Dates	moment, @vuepic/vue-datepicker
Validation	Vuetify rules + custom validators (`requiredValidator`)
Authorization	CASL (@casl/ability, @casl/vue)
Charts / calendar	ApexCharts, Chart.js, FullCalendar

Auto-imports are enabled (via `unplugin-auto-import`): common helpers like `ref`, `computed`, `onMounted`, `useI18n` are available globally without explicit import. Path aliases include `@` (src), `@core`, and `@layouts`.

3. Architecture & the Standard Module Pattern

Almost every feature is one CRUD "entity" built from **four coordinated pieces**. This repetition is the heart of the project — once you know one module, you know them all. Using *customers* as the example:

Piece	Location	Responsibility
Page	`src/pages/customers.vue`	Wires everything together; holds fetch / add / edit / delete / search / paginate logic and the API calls.
Page-config	`src/page-configs/customer.js`	Exports `breadCrumbs`, table `headers`, and shared option lists (e.g. statuses, currencies).
Stepper	`src/components/CustomerSteper/CustomerSteper.vue`	The create/edit dialog: default payload, submit (POST/PUT), `openDialog` / `openEditDialog`.
Step form(s)	`.../CustomerStep1.vue`	The actual input fields bound to `payload`, with validation.

> Note the project's spelling convention: the dialog components are called **"Steper"** (single P) throughout the codebase, e.g. `CustomerSteper`, `StuffSteper`.

4. Project Structure

```
src/
├─ pages/              One file = one route (file-based routing)
├─ page-configs/       Per-entity breadcrumbs, table headers, option lists
├─ components/
│   ├─ commons/        Shared UI: DataTable, Steper, ActionButton,
│   │                  ConfirmDialog, BreadCrumbs, ProfileSelector, DoneStep
│   └─ <Entity>Steper/ Per-entity create/edit dialog + step forms
├─ layouts/            Page shells (default with sidebar, blank) + DrawerContent (menu)
├─ plugins/            axios-plugin, authServices, vuetify, i18n, vuelidate
├─ store/              Pinia stores (authStore)
├─ router/             Router setup + global auth guard
├─ translations/       fa.js / en.js
├─ @core/ , @layouts/  Template internals & helpers (incl. utils/printRecord.js)
└─ main.js             App bootstrap
```

5. Core Conventions

API layer

All requests go through a single Axios instance (`src/plugins/axios-plugin.js`) with base URL `VITE_API_URL + 'api/'` and a Bearer token attached automatically. Endpoints are the plural kebab-case entity name. Response interceptors handle:

- **401** → toast error, log out, redirect to `/login`
- **206** → "record deleted" success toast (delete convention)
- **422 / 500** → show the server error message as a toast
- `ERR_NETWORK` → redirect to a network-error page

List endpoints return a paginated shape: `{ data: [...], last_page: N }`.

Authentication

JWT-based. On login, the token and user are stored in `localStorage` and the Pinia `authStore`. A global router guard protects any route whose meta contains `auth: true`, redirecting unauthenticated users to `/login`.

Validation & forms

Forms use Vuetify's `v-form` with field `:rules`. The shared `requiredValidator` enforces required fields. Each step form exposes a `validate()` method the stepper calls before advancing or submitting.

Localization & RTL

The whole UI is Persian and right-to-left. Labels live inline in the components; vue-i18n provides `fa.js` / `en.js` dictionaries for shared strings.

6. Reusable Building Blocks

Component	Purpose
`DataTable`	Generic table driven by `headers`. Renders `record[key]` by default, but any column can be customised via a named slot (e.g. `#status`). Includes loading state and pagination.
`Steper`	Multi-step wizard inside a dialog. Renders each step component, runs its `validate()`, moves between steps, and emits `onSubmit` on the last step. Shows a "done" step on success.
`ActionButton`	Row action buttons — edit, delete, view, copy, print — toggled by `show-*` props.
`ConfirmDialog`	Reusable yes/no confirmation (delete, restore, logout, force-delete).
`BreadCrumbs`	Page header with breadcrumb trail, a search box, "search by column" selector, refresh and "add" buttons.
`ProfileSelector` / `DoneStep`	Image/file upload control; final success screen of a stepper.

7. Feature Modules

The sidebar groups the following modules (Persian title → route):

Module (Persian)	Route	Area
داشبورد	index	Dashboard
کاربران	users	People
کارمندان	stuffs	People (staff)

حاضری کارمندان	stuff-attendance	Staff attendance
مشتریان	customers	People
مخاطبین	contacts	People
محصولات	products	Inventory
محصولات فروخته شده	product-sales	Sales
مواد خام خریداری/باقی مانده	raw-materials / remaining-raw-material	Inventory
ظرفیت تولید روزانه	production-capacities	Production
تامین کننده گان	investors	Partners
پرداخت معاشات کارمندان	stuff-payments	Finance
معاملات روزانه	daily-transactions	Finance
دریافت و پرداخت	transactions	Finance (credit/debit)
مصارف کارخانه/روزانه	office-warehouse-expense / daily-expense	Finance
حسابات بانکی	bank-accounts	Finance
دارایی ها / اموال شرکت / اموال شخصی	estates / company-tools / personal-goods	Assets
شرکت ها / گزارشات شرکت	companies / company-reports	Company
وظایف روزانه	daily-tasks	Operations
ثبت ویزای کشور ها	visa-registrations	Operations
پسورد ها	passwords	Operations

8. Special Features

Single-record print to PDF

A shared utility (`src/@core/utils/printRecord.js`) renders any single record as a branded, RTL, A4 document and opens the browser print dialog (Save as PDF or print). It is driven by the page's own `headers` , so labels stay consistent with the table. Enabled on most list pages through the print action button.

Status-change dialogs

Several modules expose their status as a coloured, clickable button in the table. Clicking it opens a small dialog to change the status and posts to a dedicated `.../change-status` endpoint. Used by **Daily Tasks** (new / in progress / done) and **Visa Registrations** (new / in progress / done / cancelled). Statuses are stored with English values and displayed with Persian labels.

Bulk staff attendance

The Staff Attendance module (حاضری کارمندان) supports a daily bulk flow: pick a date, load every staff member at once, mark each as present / absent / leave / late with an optional note, and submit them all in a single request (`bulk`

endpoint). Existing records for the chosen date are pre-filled so the day can be corrected and re-saved. Individual rows can still be edited or deleted.

Shared "description" field

Most entities carry a free-text `description` (توضیحات) field, shown as a table column and editable in the stepper form.

9. How to Add a New Module

To add a new entity (e.g. `orders`), follow the established pattern:

1. Create `src/page-configs/order.js` with `breadCrumbs`, `headers`, and any option lists.
2. Create `src/components/OrderSteper/OrderSteper.vue` (+ `OrderStep1.vue`) for create/edit, with a `defaultPayload` matching the API fields.
3. Create `src/pages/orders.vue` (copy an existing JSON-payload page such as `customers.vue`) and point the four Axios calls at the `orders` endpoint.
4. Add a `<VerticalNavLink>` entry in `src/layouts/components/DrawerContent.vue`.
5. Add custom column slots (chips, formatted dates, status buttons) on the page as needed.

> Two API styles exist: **JSON payload** (most modules send the whole `payload` object) and **multipart/FormData** (modules with file uploads, e.g. staff and users — each field is appended explicitly). Match the style of the entity you copy.

10. Running & Building

The project uses Yarn. Set `VITE_API_URL` (the Laravel API base URL) in a `.env` file first.

```
# install dependencies
yarn install

# start dev server (also builds icons)
yarn dev

# production build
yarn build

# lint
yarn lint
```

The dev and build scripts first run `build:icons` to bundle the Iconify (MDI) icon set.

> **Backend note.** This document covers the Vue frontend. It assumes a Laravel REST API exposing the matching plural endpoints (e.g. `customers`, `stuff-attendances`, `visa-registrations`, `transactions`) plus the special routes `.../change-status` and the attendance `bulk` route.